Drug review sentiment with manual LSTM cells

Advanced Natural Language Processing · Lab project · 2026/2027

Sara Baalla, Zouhair Eouichak, Malak Bartal, Aymane Youssi
25 October 2026

1. Objective and task definition

This project compares a vanilla Long Short-Term Memory network with Coupled Input-Forget Gate (CIFG) and Working Memory Connections (WMC). Each model predicts one of three sentiment categories from a patient review. All recurrent cells implement the appendix equations using PyTorch **nn.Linear**. No built-in LSTM layer implements the studied cells.

The input contains review text only. Patient ratings create the supervision: ratings 1-4 define negative, 5-6 neutral and 7-10 positive. These thresholds are a modeling choice. The labels approximate sentiment rather than providing independent linguistic annotation. Drug names, conditions, dates, ratings and helpfulness counts are excluded from model inputs.

2. Dataset and initial exploration

The Drugs.com TSV files contain 161,297 training reviews and 53,766 test reviews. The files came from the GitHub mirror cited in the references after access problems with the UCI page. The repository records SHA-256 hashes of the files actually used. We did not independently establish byte identity with a freshly downloaded UCI archive.

Raw training category	Reviews	Share
Negative	40075	24.85%
Neutral	14356	8.90%
Positive	106866	66.25%

The raw test file has 295 missing condition values, but no missing review or rating values. Since the task uses only text and rating-derived labels, missing conditions do not require dropping these reviews. The raw class distribution is strongly imbalanced, so accuracy alone can reward a classifier that predicts positive for every input.

Cleaning decisions

The pipeline decodes HTML entities, lowercases text and normalizes whitespace while preserving punctuation. It removes invalid or empty examples, then excludes identical normalized texts assigned to conflicting sentiment categories. Training cleaning removes 108 conflicting rows and 48,904 additional duplicates, leaving **112,285 unique training texts**. These decisions operate on normalized exact matches. Near-duplicates and shared drug-specific language can remain.

Data preparation and model design

3. Partitions and representation

A stratified 85/15 split with seed 42 divides the cleaned training pool into 95,443 training and 16,842 validation reviews. Exact normalized texts do not overlap. For the CPU experiment budget, fixed stratified subsets contain **12,000 training and 2,000 validation reviews**. Every architecture and setting uses these same subsets.

Experimental split	Negative	Neutral	Positive	Total
Train	2974	1068	7958	12000
Validation	496	178	1326	2000
Test	988	356	2656	4000

The vocabulary contains the 11,998 most frequent tokens from the full 95,443-review training pool, plus PAD=0 and UNK=1. Thus vocabulary fitting uses training text outside the 12,000-review optimization subset, but no validation or test text. Tokenization preserves words with internal apostrophes and separates punctuation. Each input keeps the first 128 tokens and pads shorter inputs. This limit truncates 33.36% of the full training pool.

A trainable 64-dimensional embedding converts token IDs into vectors. Hidden state h and cell memory c both have dimension 64. At each padded position the implementation preserves both previous states. The linear classifier receives the hidden state after the last useful token and produces three logits. No dropout is used in the final experiment models.

4. Manual recurrent equations

Vanilla. Input i, forget f and output o gates apply sigmoid to separate affine transforms of current embedding x and previous hidden state h. A tanh candidate g proposes new content. The updates are **c = f * c_previous + i * g** and **h = o * tanh(c)**, where * denotes elementwise multiplication. Each affine pair has one bias, attached to the input transformation.

CIFG. The input gate becomes **i = 1 - f**. Removing its learned transforms reduces parameters and couples memory retention with new input. **WMC.** The input and forget gates add tanh(Wc * c_previous), while the output gate uses tanh(Wc * c_new), as specified by the appendix. These additional full matrices have no extra biases.

Architecture	Total parameters	Recurrent cell parameters
Vanilla	801,219	33,024
CIFG	792,963	24,768
WMC	813,507	45,312

Totals include 768,000 embedding values and 195 classifier parameters. The PAD embedding row remains zero under the optimizer used here. Tests check equations against explicit matrix products, numerical gradients, finite backward gradients and invariance to trailing padding.

Experimental protocol and validation

5. Controlled training settings

Experiments run with Python 3.12.14 and PyTorch 2.14.1+cpu on CPU, with four PyTorch threads. Every run uses seed 42, four epochs, Adam and gradient clipping at norm 1.0. Class-weighted cross entropy uses weights N/(3*n_class), calculated only from the experimental training subset. All models share the vocabulary, embeddings dimension, hidden dimension, sequence limit and evaluation protocol.

The three settings are (learning rate 0.001, batch 128), (0.003, 128) and (0.001, 64). This changes one factor at a time relative to the first setting. It does not estimate the full interaction between learning rate and batch size. Weight initialization follows PyTorch defaults and remains fixed by seed. The architecture comparison reports different parameter counts rather than claiming equal model capacity.

6. Nine measured runs

Cell	Learning rate	Batch	Best epoch	Validation macro-F1
VANILLA	0.001	128	4	0.4820
VANILLA	0.003	128	4	0.5050
VANILLA	0.001	64	3	0.4817
CIFG	0.001	128	4	0.4763
CIFG	0.003	128	4	0.5246
CIFG	0.001	64	4	0.4868
WMC	0.001	128	4	0.4934
WMC	0.003	128	4	0.4843
WMC	0.001	64	3	0.4904

After each epoch, evaluation runs without gradient calculation or parameter updates. The checkpoint with the highest validation macro-F1 is retained for each configuration. Each architecture then selects its strongest configuration using validation only. Test evaluation takes place after all nine runs complete.

Metrics and interpretation

Accuracy is the fraction of correctly classified reviews. For each class, F1 = 2*TP/(actual count + predicted count). Macro-F1 averages the three class F1 values equally, assigning the neutral class the same importance as the positive class. The confusion matrix uses true classes as rows and predicted classes as columns. Ties in validation retain the first checkpoint or configuration encountered.

The highest validation macro-F1 belongs to **CIFG**: 0.5246, with learning rate 0.003 and batch size 128. This validation result selects the primary model before examining test performance.

Final test results and error analysis

7. Held-out evaluation and leakage checks

The raw test file contains 53,766 reviews. The cleaning protocol excludes 15 rows with conflicting labels, collapses exact normalized duplicates and removes 32,108 unique texts found anywhere in the full cleaned training or validation pools. This leaves 16,153 eligible test texts. A fixed stratified subset of 4,000 reviews supports the final comparison. No experimental test text overlaps those pools.

Model	LR / batch	Test accuracy	Test macro-F1
VANILLA	0.003 / 128	63.27%	0.5052
CIFG	0.003 / 128	63.78%	0.5093
WMC	0.001 / 128	62.75%	0.4858
Always positive	No training	66.40%	0.2660

The validation-selected CIFG model achieves test accuracy 63.78% and macro-F1 0.5093. The majority baseline achieves 66.40% accuracy and 0.2660 macro-F1. Comparing both metrics separates global correctness from balanced recognition of all three categories. Scores apply to this cleaned test subset, not the complete original distribution.

Confusion matrix for CIFG

Actual / predicted	Negative	Neutral	Positive
Negative	630	175	183
Neutral	128	92	136
Positive	366	461	1829

Class	Precision	Recall	F1
Negative	0.5605	0.6377	0.5966
Neutral	0.1264	0.2584	0.1697
Positive	0.8515	0.6886	0.7614

Each prediction CSV stores the actual class ID and predicted class ID in the same order as test_experience.tsv. These files support inspection of incorrect examples without changing the completed model-selection protocol. Mixed sentiment, ambiguous rating-derived labels and truncation offer hypotheses for error analysis, rather than proven causes for every mistake.

Discussion, reproducibility and references

8. Conclusions and practical limits

This implementation completes the requested manual-cell comparison. On validation, CIFG provides the strongest configuration among the nine runs. Its test macro-F1 is 0.5093. CIFG uses fewer cell parameters than Vanilla, while WMC adds direct memory-to-gate connections and increases capacity. The evidence supports a conclusion about this experiment, not universal superiority of an architecture.

The study uses a single seed and a restricted CPU budget. It reports no confidence intervals and cannot establish whether small score differences persist across initializations. Rating thresholds provide noisy sentiment labels. Positive reviews dominate, and the neutral category has fewer examples. Exact deduplication does not eliminate near-duplicates or correlations between reviews of the same drug. Truncation discards later context from long reviews.

Potential improvements

Repeat the experiment with several seeds and report mean and spread. Increase training size and sequence length, measuring computational cost as well as validation performance. Use early stopping on validation. Compare pretrained embeddings under the same protocol. Inspect neutral and mixed reviews, investigate text that explicitly mentions ratings, and consider grouping by drug when evaluating generalization. Freeze these decisions before a new held-out test evaluation.

9. Reproduction and deliverables

The public code includes the eleven numbered stages, data download helper, model tests, experiment logs, final metrics, prediction files and fixed subset manifests. The accompanying ZIP also includes the selected model checkpoints and the experimental subsets. Raw datasets and Python environments are excluded from the public code to keep the repository manageable. No credentials or machine-specific environment files belong to the submission.

Run the download helper, then stages 3, 4 and 5 to reconstruct preprocessing. Run **python -m unittest discover -s tests -v**, followed by **python etape_10_experiences.py**. Use **python etape_11_predire.py "this medicine helped me" --architecture cifg** to demonstrate inference after training. The README gives installation instructions and documents the exact scope of each stage.

Code: https://github.com/baalasara7-max/rnn-drug-review-lstm-2026

References

[1] Course brief: Guidelines_RNN_2627.pdf, Advanced Natural Language Processing, 2026/2027. Manual Vanilla, CIFG and WMC equations in the appendix.

[2] Kallumadi, S. and Grasser, F. (2018). Drug Review Dataset (Drugs.com). UCI Machine Learning Repository, dataset 462. https://doi.org/10.24432/C5SK5S

[3] Downloaded TSV mirror: https://github.com/semnan-university-ai/Drug-Review-Dataset. See resultats/synthese.json for the exact SHA-256 hashes.

[4] CIFG reading supplied in the brief: https://arxiv.org/abs/1503.04069

[5] WMC reading supplied in the brief: https://www.sciencedirect.com/science/article/pii/S0893608021003439

[6] PyTorch APIs: https://docs.pytorch.org/docs/stable/nn.html. The project uses Linear and Embedding, with manually implemented recurrence.